

Complexity_Simplified_p - Complete Reference

Puzzle 135 / secp256k1. Generated 2026-06-13.

Combines the original x-side cube-root bridge (Complexity_Simplified_p.txt) with the verified y-side quadratic bridge and N-side shadow constants.

1. Executive summary

Puzzle 135 target $x = Px^3$. The p-side file closes the x-bridge: $Px_i/rx_i = \text{Lambda} \pmod{p}$ via three cube roots n_1, n_2, n_3 of N. The y-side mirrors this with collapsed ratios CQ1, C_r1 , and lambda_y - but y has only +/- branches (no third cube slot). $\text{lambda}_y \neq \text{Lambda}$. The strongest new identity is $IP + 7 = Py^3 \pmod{p}$ where $IP = Px^1 * Px^2 * Px^3$.

2. Puzzle 135 target

```
Px3 = 9210836494447108270027136741376870869791784014198948301625976867708124077590
hex = 145d2611c823a396ef6712ce0f712f09b9b4f3135e3e0aa3230fb9b6d08d1e16
Py3 = 46351506704828816385393879789131775975171267756561783641521771795450741674800 (even y, compressed pubkey 02...)
rx3 = 26000218878731561428273279366182192513989009817816850365013828370091835863739
d band: [2^134, 2^135 - 1] and mirror [N - 2^135 - 1, N - 2^134]
```

3. Foundational parameters

```
p = 115792089237316195423570985008687907853269984665640564039457584007908834671663
N = 115792089237316195423570985008687907852837564279074904382605163141518161494337
p - N = 432420386565659656852420866390673177326
n1^3 = N (mod p) [cube roots n1,n2,n3 in source file]
```

4. X-side bridge (original, verified)

Three-way Latin square on Gx, Px, rx via n_j^{-1} . Collapsed constants:

```
CP1 = Px_i * Gx_i^{-1} mod p = 57602015833677736603574291432760600960685355547305560147555835666458430710854
CR1 = rx_i * Gx_i^{-1} mod p = 73680319372475906803320245449080571569331871474977252785503402279627244902569
Lambda = CP1 * CR1^{-1} = Px_i * rx_i^{-1} mod p = 97451685862885086182458552040892158509924235661624603229050850812487253689501
Lambda^{-1} mod p = 81068098917365752067418111170832400985023864499927122630942305971544277876663
```

```
IP = Px^1 * Px^2 * Px^3 mod p = 80184233617433755134183875136831551618578922487806929476230322368028862899162
IG = Gx^1 * Gx^2 * Gx^3 mod p = 32748224938747404814623910738487752935528512903530129802856995983256684603115
IR = rx^1 * rx^2 * rx^3 mod p = 36128738557023842197460850546205429335937024355616500847429829845960355299834
```

```
Pi = Lambda * ri (mod p) [root bridge]
IP = Lambda^3 * IR (mod p) [cubic bridge]
```

5. Y-side bridge (new, verified)

Each triple shares one $y^2 \pmod{p}$, so $Gy_1=Gy_2=Gy_3$, $Py_1=Py_2=Py_3$, $ry_1=ry_2=ry_3$ (per branch). Pubkey selects even Py . No 3-way n_j Latin square on y.

```
Gy (even) = 32670510020758816978083085130507043184471273380659243275938904335757337482424
Py (even) = 46351506704828816385393879789131775975171267756561783641521771795450741674800
ry (even, same parity as Py) = 49714739208247555872780528359092797866261457510155690641636464864972500227644
```

```
CQ1 = Py_i * Gy_i^{-1} mod p = 1020171577293033848585017509374488340661404401020742394241838048178858885772
C_r1 = ry_i * Gy_i^{-1} mod p = 1597653561656624141002295098639632098151602281267190861934017947166836383973
lambda_y = CQ1 * C_r1^{-1} = Py_i * ry_i^{-1} mod p = 92736738943421429813433900502071579205213592459201379042094542895571506924317
lambda_y^{-1} mod p = 47301002286168168178041007158604439612095367231870254795510002266969134325448

lambda_y / Lambda mod p = 96896380249913414475490055076671490450009407789240445582882446462268000724666
lambda_y^2 mod p = 38266475446343285184463878434166515158578856436171244279272671675578609516352
(Px^3+7)/(rx^3+7) mod p = 38266475446343285184463878434166515158578856436171244279272671675578609516352
```

6. Key identity: $IP + 7 = Py^3$

```
IP = Px^1 * Px^2 * Px^3 mod p = 80184233617433755134183875136831551618578922487806929476230322368028862899162
IP + 7 mod p = 80184233617433755134183875136831551618578922487806929476230322368028862899169
```

Py3^2 mod p = 80184233617433755134183875136831551618578922487806929476230322368028862899169
Match: True

This ties the x-product block directly to the puzzle pubkey y-coordinate. It is the main structural link between the x-side ladder and the target point.

7. X-side vs Y-side comparison

Layer	x-side	y-side
Root degree	x^3 -> 3 cube roots n1,n2,n3	y^2 -> 2 branches (+/- only)
Normalize	n_j^-1 Latin square A/B/C	branch pick only (no n_j on y)
Collapse G	CP1 = Px/Gx	CQ1 = Py/Gy
Collapse r	CR1 = rx/Gx	C_r1 = ry/Gy
Bridge	Lambda = Px/rx	lambda_y = Py/ry (!= Lambda)
Power law	Lambda^3 cubic	lambda_y^2 quadratic
Triple product	IP, IG, IR	IPy, IGy, IRy
Puzzle 135	Px3	Py3 even branch

8. Branch grid (y only)

Py+ / ry+ or Py- / ry- (same parity) => lambda_y =
92736738943421429813433900502071579205213592459201379042094542895571506924317
Py+ / ry- or Py- / ry+ (mixed parity) => lambda_y =
230553500293894765610137084506616328648056392206439184997363041112337327747346

Odd branches:

Gy- = 83121579216557378445487899878180864668798711284981320763518679672151497189239
Py- = 69440582532487379038177105219556131878098716909078780397935812212458092996863
ry- = 66077350029068639550790456649595109987008527155484873397821119142936334444019

9. N-side shadow (for Complexity_Simplified_N)

Lambda_N = Px3*rx3^-1 mod N = 107329397079532295557318141750802481743365519021307735330654746890138631346025
lambda_y_N = Py3*ry3^-1 mod N = 88705711692776652458910848945234273793234577019423001147270510337199062027057
GAP_x = Lambda_N - Lambda mod N = 9877711216647209374859589709910323233441283359683132101603896077651377656524
GAP_y = lambda_y_N - Lambda_N mod N = 97168403850560552325163692203119699902706622277190170199220926588578592175369

Reflection: map_p_to_n(y) = floor(N*y/p) mod N
map_p_to_n(Py3) + map_p_to_n(-Py3) = 115792089237316195423570985008687907852837564279074904382605163141518161494336 (= N-1)

Do NOT copy Lambda to N blindly. Use Lambda_N = Lambda + GAP_x. Y-side needs separate lambda_y_N and reflection pairing.

10. Verification checklist

- [PASS] shared_y2_px
- [PASS] shared_y2_gx
- [PASS] shared_y2_rx
- [PASS] ip_plus_7
- [PASS] lam_y_sq
- [PASS] cq1_const
- [PASS] c_r1_const
- [PASS] lam_y_const
- [PASS] py_on_curve
- [PASS] refl_py
- [PASS] refl_ry

11. What this does / does not give

DOES: complete p-side geometry for Puzzle 135; second bridge constant λ_y ; $IP+7=Py3^2$; N-side GAP and reflection inputs.

DOES NOT: yield private key d directly. Scalar recovery still requires ECDLP (kangaroo on almost.txt start point) or ladder narrowing in $[2^{134}, 2^{135})$.

12. Source files

Complexity_Simplified_p.txt (this note, $x+y$)

Complexity_Simplified_N.txt (N-side, needs λ_N + range-variable δ)

almost.txt (EC trace $\rightarrow$ kangaroo on start point)

primes_public_all_puzzles.txt (public factorizations, no private key for 135)